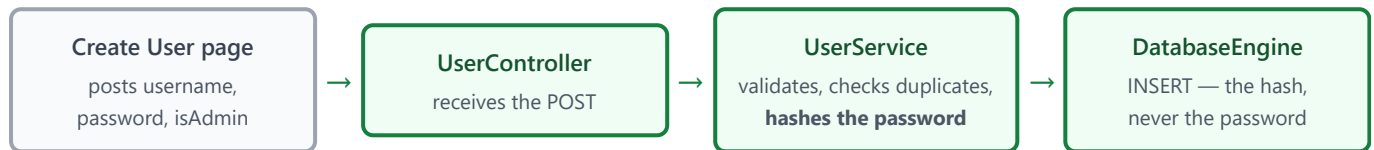


Creating Users — with Hashed + Salted Passwords

Build the Create User page's backend, and store passwords the only acceptable way: salted, hashed, never readable. Same layering, same bare flow as Parts 1–2.

0. What you're building



Prerequisites: Part 1's backend exists (the `User` model, `IUserService / UserService`, `DatabaseEngine.GetUserByUsername`). The page itself is already in the project at `Views/User/Index.cshtml`, and the Quotes page has a *Create User* button linking to `/User`. The view defines your controller's contract: **GET** `/User` returns the page; **POST** `/User` receives JSON `{ "username": "...", "password": "...", "isAdmin": 1 or 0 }` and answers 2xx on success or 400 with a message (shown in the page's red alert box).

Access control on this page comes later — deliberately

Right now anyone can open `/User`. That's your explicit plan (role-based access is your next exercise), and it's also what makes the build order work: you can create your first properly-hashed users before login enforcement exists. When you wire up your role checks, this page is a one-liner to protect — the button on the Quotes page carries a `TODO` comment marking the spot, and the controller gets the same treatment as your other admin-only endpoints.

0.5 Hashing & salting — the five-minute version

Why storing passwords is forbidden: if your database is ever stolen (SQL injection, leaked backup, disgruntled insider), a plain-text Password column hands the attacker every account — and because people reuse passwords, their email and banking too. The fix is to store something that *proves* the password without *containing* it.

A hash is a one-way computation. The same input always produces the same output, but there is no way to run it backwards — from the output you cannot recover the input. So instead of asking "does the typed password equal the stored password?", login asks "does the typed password *hash* to the stored value?". The real password is never stored, never compared, never recoverable.

The salt fixes the two holes hashing alone leaves open. If hashing were all we did, two users with password `1234` would have identical hashes (crack one, you've cracked both — and you've revealed they share a password), and attackers could use precomputed tables of common passwords' hashes ("rainbow tables") to crack most of a stolen database in seconds. A **salt** is a random value generated per user and mixed into the hash computation. Same password, different salt → completely different hash. Every user's hash must now be attacked individually, from scratch.

Proof from your own seed script — both of these are the password `1234` :

```
admin:  AQAAAAIAAYagAAAAEGwPdM5tU03R1DrkHbjr8ekmXADF1k0Ja8DKDC/8Vde1iKZAA/DfWTax+9gJDRviTA==
user:   AQAAAAIAAYagAAAAENDIEU1HLKbTKS//pgahV0wUjhJGSZTWaIPfk/9Hi1lwoUVMhUfmZxRenU9K346+7A==
```

One more trick: deliberate slowness. Password hashers (this one is PBKDF2) repeat the computation ~100,000 times. You log in once, so 50ms is nothing; an attacker guessing billions of passwords pays 50ms *per guess*. Slowness is the feature.

What's inside that base64 blob

ASP.NET Core's `PasswordHasher` packs everything verification needs into the one stored string: a format marker, the algorithm + iteration count, **the salt**, and the derived key. That's why there is no separate Salt column — the salt rides inside the hash value. At login, `VerifyHashedPassword` unpacks the salt and settings, re-runs the identical computation on the typed password, and compares results (Part 1, Step 4).

Never invent your own

No SHA256-of-the-password, no homemade salting schemes, no "encryption" (encryption is reversible — wrong tool). `PasswordHasher<T>` ships with the framework, implements PBKDF2 correctly, and upgrades its parameters over time. Using it is one line to hash and one to verify.

Step 1 — Reuse the User model (no separate request model)

The POST body binds straight to the `User` model you already built in Part 1 — there is no `CreateUserRequest` class. For reference, that model, with `IsAdmin` as an `int`:

`Models/User.cs` (already exists — no new file)

```
public class User
{
    public int UserId { get; set; }
    public string Username { get; set; }
    public string Password { get; set; }
    public int IsAdmin { get; set; } // 1 = admin, 0 = regular user
}
```

What binds, and the trade-off: the view posts `{ username, password, isAdmin }` where `isAdmin` is `1` or `0`; those three bind to the matching properties and `UserId` is left unset. Reusing the whole model instead of a dedicated input class is fine here — but know exactly *why* it's safe, field by field:

- **UserId** — even if a crafted request set it, the INSERT omits it (the column is `IDENTITY`, the database assigns it), so it can't do harm.
- **Password** — arrives as plain text, then the service overwrites it with the salted hash before anything is stored (Step 3).
- **IsAdmin** — is *meant* to be chosen by the caller here (an admin picking the new user's role), so exposing it is correct — the exact opposite of login, where a visitor must never set their own role.

(The stricter alternative — a separate request class exposing only the fields a caller may send — earns its keep on endpoints whose model carries sensitive fields the client must never touch. For this create, where every posted field is either meant to be set or harmlessly ignored, reusing `User` is the simpler, reasonable choice.)

Step 2 — Add the INSERT to DatabaseEngine

`Data/DatabaseEngine.cs` (add below `GetUserByUsername`)

```

public static void CreateUser(User user)
{
    string query = @"
        INSERT INTO Users
        (
            Username,
            Password,
            IsAdmin
        )
        VALUES
        (
            @Username,
            @Password,
            @IsAdmin
        )";

    using var command = new SqlCommand(query);

    command.Parameters.AddWithValue("@Username", user.Username);
    command.Parameters.AddWithValue("@Password", user.Password); // by the time it gets here,
this is the HASH
    command.Parameters.AddWithValue("@IsAdmin", user.IsAdmin);

    ExecuteNonQuery(command);
}

```

What it does & why: same shape as `CreateQuote`: parameterised INSERT, executed through your existing `ExecuteNonQuery`. The engine neither knows nor cares that the Password value is a hash — hashing happened a layer up (Step 3), and by design nothing below the service ever sees a real password. `UserId` is omitted because the column is `IDENTITY`: the database assigns it.

Step 3 — Extend the service: validate → check duplicate → hash → insert

Interfaces/IUserService.cs (add one method)

```

public interface IUserService
{
    User ValidateUser(string username, string password);

    void CreateUser(User user); // ← add (takes the whole User)
}

```

Services/UserService.cs (add the method; add the using at the top)

```

using Microsoft.AspNetCore.Identity;    // ← add: PasswordHasher lives here

public void CreateUser(User user)
{
    if (string.IsNullOrEmpty(user.Username))
        throw new ArgumentException("Username is required.");

    if (string.IsNullOrEmpty(user.Password))
        throw new ArgumentException("Password is required.");

    if (DatabaseEngine.GetUserByUsername(user.Username).Rows.Count > 0)
        throw new ArgumentException("Username is already taken.");

    // The plain-text password's life ends on this line: user.Password goes in
    // as text and comes back out as a salted hash, overwriting itself. Nothing
    // below — not the engine, not the database — ever sees the real password.
    var hasher = new PasswordHasher<User>();
    user.Password = hasher.HashPassword(user, user.Password);

    DatabaseEngine.CreateUser(user);    // user.IsAdmin (int) rides along untouched
}

```

What it does & why, in order:

- **Input checks first** — same style as your other services; read the fields off the passed-in `user`, and bad input never reaches the database.
- **Duplicate check** reuses `GetUserByUsername` from Part 1 — no new query needed. It exists to give a *friendly* error. The real guarantee is the `UNIQUE` constraint on the column: if two requests raced past this check with the same name, the database would still reject the second INSERT (as a 500 — rare enough not to special-case while learning).
- `HashPassword(user, user.Password)` generates a fresh random salt, runs PBKDF2, and returns the single blob (salt included) that *overwrites* `user.Password` before storage. Each call salts anew — create two users with the same password and watch the stored values differ. (The `user` argument exists so custom hashers *could* use per-user data; the default hasher ignores it.)
- **IsAdmin is never touched here** — it rides along inside the `user` object as the `int` the client posted (1 or 0), straight through to the INSERT. No conversion, because there is no bool anywhere in the chain.
- **Why hash in the service and not the controller or engine:** the service is your business-rules layer, and "passwords are stored hashed" is a business rule. Controller stays thin, engine stays dumb, and there is exactly one line in the whole codebase where a plain-text password exists on its way to storage.

Step 4 — Create the UserController

Controllers/UserController.cs (new file)

```
using Microsoft.AspNetCore.Mvc;
using QuotesProject.Interfaces;
using QuotesProject.Models;

namespace QuotesProject.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class UserController : Controller
    {
        private readonly IUserService _userService;

        public UserController(IUserService userService)
        {
            _userService = userService;
        }

        [HttpGet]
        public IActionResult Index()
        {
            return View(); // serves Views/User/Index.cshtml
        }

        [HttpPost]
        public IActionResult CreateUser(User user)
        {
            if (!ModelState.IsValid)
                return BadRequest(ModelState);

            try
            {
                _userService.CreateUser(user);
                return Ok();
            }
            catch (ArgumentException ex)
            {
                return BadRequest(ex.Message); // missing fields / duplicate username
            }
            catch (Exception ex)
            {
                return StatusCode(500, "Error creating user: " + ex.Message);
            }
        }
    }
}
```

What it does & why: the same controller shape as everything else in your project — `[ApiController]` binds the JSON body to the `User` model (the posted `isAdmin` value, `1` or `0`, lands in the `int IsAdmin` property), the service does the thinking, and exceptions map to status codes (`ArgumentException` → 400 with the message the page displays; anything else → 500). Routes come out as **GET /User** and **POST /User**, which is exactly what the view's `fetch` calls. No new registration needed in `Program.cs` — `IUserService` was registered in Part 1, and controllers are discovered automatically.

When you add your role-based access later

This controller is admin-only territory. Once your role checks exist (however you design them), protect it exactly like your other admin endpoints, and on the Quotes page wrap the Create User button in your admin check — the `<!-- TODO -->` comment above the button marks the spot. Until then, remember the page is open.

Step 5 — The page and the button (already in the project)

Nothing to build here — just know what's there:

- `Views/User/Index.cshtml` — the Create User form: username, password, an *Admin* checkbox (the script posts its checked state as `isAdmin: 1` or `0`, binding straight to the model's `int`), a green success alert, a red error alert that shows whatever text your controller returns with a 400 (so "Username is already taken." appears right on the form), and a *Back to Quotes* link.
- `Views/Quote/Index.cshtml` — a *Create User* button next to the "Quotes List" heading, linking to `/User`, with the TODO comment for your future admin gating.

Step 6 — The seed script now stores hashes too

`Scripts/CreateUsersTable.sql` has been updated: `admin` and `user` still log in with `1234`, but the Password column now holds the salted PBKDF2 hashes (the two blobs shown in Section 0.5). The script is safe to re-run — if the users already exist with plain-text passwords from earlier, it *updates* them to the hashes. Run it once before testing.

Hash storage and hash verification must ship together

A hashed password in the database can never equal what the user types, so plain-text comparison in `ValidateUser` would reject every login. Part 1's Step 4 (updated) contains the matching verification code — `VerifyHashedPassword`. If logins suddenly all fail with "Invalid username or password", the near-certain cause is hashed storage + plain-text comparison (or the reverse: you re-ran the old seed script and got plain text back).

Step 7 — Test it

Do this	You should see
1. On <code>/Quote</code> , click <i>Create User</i>	The <code>/User</code> page loads.
2. Create <code>dylan</code> / <code>abcd</code> , Admin unchecked	Green success message; form clears.
3. In SSMS: <code>SELECT Username, Password, IsAdmin FROM Users</code>	<code>dylan</code> 's Password is an <code>AQAAAA...</code> blob (~84 chars), IsAdmin = 0. The string <code>abcd</code> appears nowhere.
4. Create <code>test</code> with the <i>same</i> password <code>abcd</code>	A <i>different</i> <code>AQAAAA...</code> blob — the salt at work, live in your own table.
5. Try to create <code>dylan</code> again	Red alert: "Username is already taken." (HTTP 400).
6. Log in as <code>dylan</code> / <code>abcd</code> (Part 1 with updated Step 4)	Login works — verification hashed your typed password with the stored salt and matched.
7. Log in as <code>dylan</code> / <code>abce</code>	401, same "Invalid username or password." as ever.

Done

Users are now created through the app, roles assigned at creation, and no readable password exists anywhere in your system — not in the database, not in the logs, not in the code. When test #4 shows two different blobs for the same password, you've *seen* salting rather than read about it.